

TAXIDJIBOUTI

Cadrage du 1^{er} MVP — Backend

Périmètre fonctionnel, état du code et plan d'exécution

Architecture : .NET · Clean Architecture · PostGIS · SignalR · Azure Container Apps

Date de l'audit : 24 juin 2026

Principe directeur : YAGNI — rien hors du strict nécessaire au premier lancement réel.

Sommaire

Comment lire ce document

Ce document fusionne le cadrage produit et un audit factuel du code (handlers, endpoints et entités réellement présents au 24 juin 2026). Il liste l'ensemble des features du périmètre MVP, leur raison d'être, leur état, leur priorité et leur emplacement dans la Clean Architecture.

Légende — état	Légende — priorité
✓ Livré	● P0 — bloquant lancement
🔧 À faire	● P1 — juste après lancement
🧩 Partiel	● P2 — post-MVP

1. Vision du MVP

Permettre à un **client** de commander un taxi/VTC à Djibouti, à un **chauffeur vérifié** de recevoir et d'effectuer la course, et à un **admin** de superviser la plateforme — du bout en bout, de manière **fiable** (joignabilité hors-app), **sûre** (chauffeurs approuvés) et **traçable** (tarif final, paiement).

Décisions produit actées

- **Paiement : cash uniquement** au lancement. L'enum `PaymentMethod` est prêt pour le futur ; l'intégration PSP (D-Money) est reportée en P2.
- **Onboarding chauffeur : validation manuelle par l'admin** — documents vérifiés hors-app au lancement, upload in-app en P1.2.

Hors périmètre MVP (assumé)

- Paiement électronique réel (D-Money) — Djibouti est majoritairement **cash**.
- Pagination, multi-tenant, cache distribué (Redis) — non requis au lancement.
- Frontend (React, hors dépôt).

2. État de l'existant (déjà livré — vérifié dans le code)

La plomberie Clean Architecture est posée et la majorité du « happy path » est fonctionnelle. Le tableau ci-dessous recense ce qui est confirmé livré.

Module	Feature	État
Identity	Inscription par numéro de téléphone	✓
Identity	Connexion (JWT maison via ITokenService)	✓
Identity	Refresh tokens (rotation, FamilyId, détection de réutilisation, hash SHA256)	✓
Identity	Révocation de token + nettoyage en tâche de fond	✓
Identity	Account lockout (tentatives échouées)	✓
Identity	GetMe (profil courant)	✓
Pricing	Estimation de prix par zones (fallback 1000 FDJ)	✓
Drivers	Création / mise à jour du profil chauffeur	✓
Drivers	Bascule de disponibilité	✓
Drivers	Position GPS temps réel (PostGIS geography(Point,4326))	✓
Drivers	Note moyenne (recalculée à chaque notation)	✓
Rides	Demande de course (Request)	✓
Rides	Cycle de vie complet Pending → Offered → Accepted → DriverArrived → InProgress → Completed	✓
Rides	Annulation client / chauffeur	✓
Rides	Notation (Rate, 1-5 + commentaire) + recalcul de la note moyenne chauffeur	✓
Rides	Signalement (Report, motif + description)	✓
Rides	Mes courses (GetMyRides) / courses en attente	✓
Dispatch	Matching de proximité PostGIS (IDriverLocator, rayon 5 km, max 20)	✓
Dispatch	Attribution en vagues (min(3, candidats), TTL 15 s, premier-arrivé-gagne)	✓
Dispatch	Verrou de concurrence optimiste (xmin) → 409 sur acceptation concurrente	✓
Dispatch	Relance de vague à l'expiration (OfferTimeoutService, poll 5 s)	✓
Dispatch	Révocation des offres perdantes / à l'expiration / à l'annulation client	✓
Realtime	SignalR RideHub (groupes ; rideOffered, rideStatusChanged, driverLocationUpdated, rideOfferRevoked)	✓
Administration	Statistiques + listes (utilisateurs, chauffeurs, courses, signalements) — lecture seule	✓
Transverse	GlobalExceptionHandler, SecurityHeadersMiddleware (OWASP)	✓
Transverse	Logging source-generated + OpenTelemetry → dashboard Aspire	✓

Module	Feature	État
Transverse	Documentation API Scalar (/scalar)	✓

Entités du Domain (réelles)

Entité	Fichier	Propriétés clés
ApplicationUser	Identity/ApplicationUser.cs	FullName, PhoneNumber, CreatedAt (+ Identity)
RefreshToken	Identity/RefreshToken.cs	UserId, TokenHash, ExpiresAt, IsRevoked, FamilyId, ReplacedByTokenId
Driver	Drivers/Driver.cs	UserId, LicenseNumber, VehiclePlate, VehicleType, IsAvailable, AverageRating, LastLocation (Point), LastLocationAt
Ride	Rides/Ride.cs	ClientId, DriverId?, Pickup/Destination (Address/Zone/Lat/Lon), EstimatedPrice, Status, AcceptedAt, CompletedAt, OfferedDriverIds, TriedDriverIds, OfferExpiresAt
Rating	Rides/Rating.cs	RideId, ClientId, DriverId, Score (1-5), Comment
Report	Rides/Report.cs	RideId, ClientId, DriverId?, Reason, Description
ZonePrice	Pricing/ZonePrice.cs	FromZone, ToZone, Price (défaut 1000 FDJ)

Endpoints exposés (réels)

Identity

POST /api/auth/register
 POST /api/auth/login
 POST /api/auth/refresh
 POST /api/auth/revoke
 GET /api/auth/me

Drivers

POST /api/drivers (upsert profil)
 POST /api/drivers/set-availability
 GET /api/drivers/me

Rides

POST /api/rides/request
 POST /api/rides/{id}/accept (acceptation directe)
 POST /api/rides/{id}/arrived
 POST /api/rides/{id}/start
 POST /api/rides/{id}/complete
 POST /api/rides/{id}/cancel
 POST /api/rides/{id}/rate
 POST /api/rides/{id}/report
 GET /api/rides/my-rides
 GET /api/rides/pending

Dispatch

GET /api/dispatch/nearest-drivers (Admin)
 POST /api/rides/{id}/accept-offer
 POST /api/rides/{id}/decline-offer

Pricing

GET /api/pricing/estimate

Admin (lecture seule)

GET /api/admin/stats

GET /api/admin/users

GET /api/admin/drivers

GET /api/admin/rides

GET /api/admin/reports

Realtime (SignalR) — hub /hubs/ride

Méthodes : JoinDriversGroup, JoinMyDriverGroup, JoinAdminsGroup, JoinClientGroup, JoinRideGroup, SendDriverLocation

Événements : rideOffered, rideOfferRevoked, rideStatusChanged, newPendingRide, driverLocationUpdated

3. Features du MVP à implémenter

● P0 — Bloquants pour un lancement crédible

P0.1 Vérification & approbation chauffeur (KYC) 🛠️ À faire

Raison d'être — Aujourd'hui Driver.Create ne porte aucun statut d'approbation (l'entité Driver n'a que IsAvailable). N'importe quel compte peut renseigner un profil chauffeur et recevoir des courses immédiatement — trou de sécurité/confiance inacceptable pour transporter des passagers. C'est la feature qui sépare un POC d'un service laçable, et le préalable à la décision « validation manuelle admin ».

Périmètre

- Statut chauffeur dans l'agrégat Domain : PendingApproval | Approved | Suspended | Rejected.
- Méthodes riches Approve() / Suspend() / Reject() → Result.
- Garde métier CanReceiveRides (approuvé ET disponible).
- Filtrage du dispatch : IDriverLocator ne retourne que les chauffeurs approuvés (ajouter le statut au filtre existant IsAvailable + fraîcheur).
- Endpoints Admin : ApproveDriver, SuspendDriver (module Administration, aujourd'hui lecture seule).

Emplacement — Taxi.Domain/Drivers · Taxi.Application/Administration · Taxi.Infrastructure/Dispatch/DriverLocator.cs · Taxi.Web.Api/Modules/Admin

Critères d'acceptation

- Un chauffeur fraîchement inscrit est PendingApproval et n'apparaît jamais dans une vague de dispatch.
- Seul un Admin peut approuver / suspendre.
- Un chauffeur suspendu cesse immédiatement de recevoir des offres.

P0.2 Notifications push (FCM) hors connexion 🛠️ À faire

Raison d'être — SignalR est in-process et exige une connexion WebSocket active (seul RideHub in-process, aucune dépendance push). Un chauffeur dont l'app est en arrière-plan ne reçoit jamais l'offre → le dispatch en vagues tourne dans le vide. Sans push, tout le système de dispatch est inopérant en conditions réelles.

Périmètre

- Abstraction IPushNotifier en Application (même pattern que IRealtimeNotifier / IDriverLocator).
- Implémentation FCM en Infrastructure (HttpClient).
- Stockage du DeviceToken (sur ApplicationUser ou table DeviceTokens).
- Dispatch double canal : SignalR (fluidité in-app) + push (réveil hors-app), branché dans RideDispatcher à l'émission de l'offre.

Emplacement — Taxi.Application/Realtime · Taxi.Infrastructure (impl FCM) · Taxi.Application/Dispatch/RideDispatcher.cs

Critères d'acceptation

- Un chauffeur app fermée reçoit une notification d'offre.

- L'échec d'envoi push ne casse pas le flux de dispatch (best-effort, loggé) — cohérent avec le try/catch déjà en place dans SignalRRealtimeNotifier.

P0.3 Statut terminal « aucun chauffeur trouvé » À faire

Raison d'être — Bug de cycle de vie confirmé. À l'expiration d'une vague, OfferTimeoutService rappelle DispatchAsync. Si tous les candidats sont déjà dans TriedDriverIds, RideDispatcher notifie les admins (NewPendingRideAsync) mais la course reste en Pending sans fin et le client n'est jamais informé qu'aucun taxi n'est disponible. Une course doit pouvoir échouer proprement.

Périmètre

- Nouveau statut RideStatus.NoDriverFound.
- Compteur de vagues WaveCount + plafond MaxWaves dans l'agrégat Ride.
- Méthode MarkNoDriverFound() → Result.
- Notification client lorsque la course est abandonnée (nouvel usage de IRealtimeNotifier).

Emplacement — Taxi.Domain/Rides · Taxi.Application/Dispatch/RideDispatcher.cs · Taxi.Application/Realtime

Critères d'acceptation

- Après MaxWaves sans candidat, la course passe en NoDriverFound et le client est notifié.
- Plus aucune boucle infinie de dispatch.

● P1 — Nécessaires pour boucler l'expérience

P1.1 Tarif final + mode de paiement (cash) 🛠️ À faire

Raison d'être — Une course se termine sur un montant réel, pas seulement une estimation. Aujourd'hui Ride ne porte que EstimatedPrice → impossible de produire un CA fiable ou de réconcilier. Le marché étant cash, on fige le tarif final et le mode de paiement à la complétion.

Périmètre

- Ride.FinalPrice (figé à la complétion) + Ride.PaymentMethod (enum { Cash, DMoney }).
- Complete(decimal finalPrice) fige le montant (la signature actuelle Complete() est sans paramètre).
- Remontée du FinalPrice dans les stats Admin (GetAdminStatsQueryHandler).
- Exposition du montant dû dans le RideDto pour affichage client + chauffeur.

Emplacement — Taxi.Domain/Rides ·

Taxi.Application/Rides/Transitions/CompleteRideCommandHandler.cs ·

Taxi.Application/Administration/Stats

P1.2 Upload des documents chauffeur (Azure Blob) 🛠️ À faire

Raison d'être — Complète P0.1 — au lancement la vérification est manuelle hors-app ; cette feature permet à l'admin de voir permis et carte grise in-app pour approuver en connaissance de cause (« Identité Phase 3 » de la roadmap).

Périmètre

- Abstraction IDocumentStorage en Application.
- Implémentation Azure Blob Storage en Infrastructure.
- Endpoint d'upload chauffeur + exposition des liens côté Admin (sur l'écran d'approbation P0.1).

Emplacement — Taxi.Application (abstraction) · Taxi.Infrastructure (impl Blob) ·
Taxi.Web.Api/Modules/Drivers + Modules/Admin

P1.3 Vérification du numéro de téléphone (OTP SMS) 🛠️ À faire

Raison d'être — L'inscription crée un compte sur un numéro non vérifié (RegisterCommandHandler ne vérifie pas le numéro). À Djibouti, le téléphone EST l'identité : un numéro non vérifié = comptes fantômes et impossibilité de rappeler le client. OTP SMS à l'inscription.

Périmètre

- Abstraction ISmsSender en Application.
- Implémentation fournisseur SMS en Infrastructure.
- Flux OTP : génération + vérification du code à l'inscription (code hashé, expiration courte).
- **Dépendance externe à trancher** : fournisseur SMS à Djibouti (Djibouti Telecom / agrégateur régional) — coût + intégration. L'abstraction permet de lancer le développement sans bloquer sur ce choix.

Emplacement — Taxi.Application/Identity · Taxi.Infrastructure (impl SMS)

P1.4 Gestion des tarifs par zone (CRUD admin) À faire

Raison d'être — Le pricing existe en lecture seule avec fallback à 1000 FDJ (seul EstimatePriceQueryHandler existe, aucun CRUD). Sans interface admin pour définir les vrais tarifs entre zones de Djibouti, toutes les courses coûtent 1000 FDJ — irréaliste. C'est ce qui rend le prix crédible.

Périmètre

- Commands admin sur ZonePrice : créer / modifier / supprimer + liste.
- Réservé au rôle Admin (réutilise le pattern Spec/handler existant).

Emplacement — Taxi.Application/Pricing (commands) · Taxi.Web.Api/Modules/Admin ou Modules/Pricing

P1.5 Motif d'annulation À faire

Raison d'être — L'annulation existe mais sans motif (CancelByClient() / CancelByDriver() ne capturent rien). Pour les litiges (fréquents en mobilité), la modération et les futures statistiques, il faut tracer qui annule et pourquoi. Évite les abus côté client comme chauffeur.

Périmètre

- CancellationReason (enum + texte libre optionnel) capturé dans les transitions d'annulation.
- Remontée dans les listes Admin.

Emplacement — Taxi.Domain/Rides ·
Taxi.Application/Rides/Cancel/CancelRideCommandHandler.cs

 P2 — Post-MVP (différable)

Feature	Raison du report
Intégration paiement D-Money réelle	L'enum PaymentMethod est prêt ; cash suffit au lancement
Modération admin avancée (ban/suspension utilisateur)	P0.1 couvre déjà la suspension chauffeur ; la suspension client suit
Réinitialisation du mot de passe	Dépend du canal OTP (P1.3) ; à enchaîner juste après
Historique / reçus de course détaillés	Confort, non bloquant
Annulation avec frais (no-show, pénalités)	Politique commerciale à stabiliser d'abord (P1.5 pose déjà le motif)
Pagination des listes Admin	À introduire quand le volume le justifie
Notifications SMS transactionnelles (au-delà OTP)	Optimisation rétention
Surge pricing / pooling / ETA-maps	Optimisations post-traction (spec wave dispatch déjà en place)

4. Ordre d'exécution recommandé

P0.1 (KYC) → P0.3 (NoDriverFound) → P0.2 (Push FCM) → P1.1 (Paielement cash) → P1.4 (CRUD tarifs) → P1.5 (Motif annulation) → P1.2 (Documents Blob) → P1.3 (OTP SMS)

Justification

- **P0.1 et P0.3 sont des changements de domaine purs** — rapides, testables en isolation, zéro dépendance externe. On durcit le cœur métier d'abord.
- **P0.2 (FCM)** introduit la première dépendance externe et profite d'un domaine déjà stable.
- **P1.1 / P1.4 / P1.5** sont des ajouts de domaine/CRUD triviaux une fois le cœur touché, et rendent prix + paiement + litiges crédibles.
- **P1.2 / P1.3** ajoutent les dépendances externes restantes (Blob, SMS) — les plus longues à câbler côté fournisseur.

5. Garde-fous Clean Architecture

À respecter pour chaque feature implémentée :

- **Dépendances inward** : toute dépendance externe (FCM, Blob, SMS) passe par une abstraction en Application, implémentée en Infrastructure — jamais de SDK externe ni de DbContext qui fuit vers l'intérieur.
- **Domaine riche** : statuts et transitions (Approve, MarkNoDriverFound, Complete(finalPrice)) vivent dans les agrégats et retournent Result — pas de logique métier dans les services (anti-anémique).
- **CQRS maison** : commands/queries + handlers injectés directement — jamais MediatR.
- **Result pattern** : aucune exception pour le flow métier ; mapping HTTP via ToHttpResult().
- **Testabilité** : dépendances injectées, chaque transition couverte par un test xUnit (cohérent avec la suite existante — 99 tests verts).
- **Logging** : décisions métier uniquement, [LoggerMessage] source-generated, pas de re-log du cycle de vie.

6. Définition de « MVP terminé »

Le MVP backend est prêt à lancer lorsque :

- ☐ Un client peut commander une course et est toujours informé du résultat (chauffeur trouvé ou NoDriverFound).
- ☐ Seuls des chauffeurs approuvés reçoivent des courses.
- ☐ Les chauffeurs reçoivent les offres app fermée (push FCM).
- ☐ Chaque course terminée porte un tarif final et un mode de paiement (cash).
- ☐ L'admin peut définir les tarifs par zone (plus de prix unique à 1000 FDJ).
- ☐ Toute annulation porte un motif.
- ☐ Tous les nouveaux comportements sont couverts par des tests.
- ☐ dotnet build et dotnet test passent au vert.